05. Agent Instruction Evals

#Shivanitha

Agent Instruction Eval

- Testing whether an AI agent follows its instructions & produces expected behaviour.
- Agent Eval = Expected behaviour → Test → Measure → Improve.
- Not just the final answer, also evaluates actions, tool use, decisions, and constraints.

Why needed?

- Agent performs multiple steps, so a correct final answer can still come from wrong behaviour.

Eg: correct weather answer + no weather-tool usage → Final Ans ✓, Agent behaviour ✗

Prompt eval v/s Agent eval

Prompt eval	Agent eval
→ Mainly evaluates output	→ evaluates behaviour + output
→ Usually input ⇒ output	→ can involve many steps
→ Response quality.	→ instructions, actions, tools, result.

Remember: Prompt eval ⇒ Is the output good?
Agent eval ⇒ Did it behave as instructed & complete the task?

Expected Behaviour

→ Before testing, define what the agent should do.
→ Eg: Receive Order ID → Call DB → Use correct ID → Read Result → Tell user status.

This becomes the evaluation reference.

Main evaluation Dimensions

- Instruction following → followed instructions?
- Task completion → achieve goal?
- Tool usage → Correct tool?
- Tool arguments → correct parameters?
- Decision / action → correct next action?
- Output correctness → correct final answer?
- Constraint adherence → Stayed within limits?

Task Completion ≠ Instruction Following

An agent can : ⊙ complete task ✓ but violate instructions ✗
⊙ follow instructions ✓ but fail task ✗
Good agent = both satisfied

Evaluation Criteria

— Use measurable criteria, not 'agent seems good'.
— eg : Correct tool ✓ correct input ✓ Authorization ✓ Privacy ✓
Correct answer ✓
Makes evaluation objective + repeatable.

Evaluation Methods

— PASS / FAIL : strict requirements
eg : Required to DB call made → PASS,
guessed answer → FAIL

— Rubric / Score : 0 → Failed 1 → Partially Followed
2 → Mostly Followed 3 → Fully Followed

useful for complex behaviour.

Test Case

Structure : Input / Goal → Instructions → Expected Behaviour →
→ Actual Behaviour → Evaluation.

Test cases should include : ⊙ Normal cases ⊙ Edge cases
⊙ Invalid Inputs ⊙ Tool failures ⊙ Instruction conflicts
⊙ Security / prompt-injection cases.

Happy Path V/s Edge Case

Happy : Normal valid situation
Edge cases : missing info, invalid input, Tool unavailable,
empty result, conflicting information.
also evaluate how the agent fails
Tool unavailable → report failure, don't invent a result.

Tool-Use Evaluation

check : ① Tool selection → right tool?
② Arguments → right parameters?
③ Sequence → correct order?
④ Result handling → interpreted result correctly?
⑤ Unnecessary calls → avoided extra tools?

correct answer without a required tool call can still be an instruction-following failure.

Instruction Hierarchy

Typical priority : System → Developer / Application → User → Tool / External data.

when instructions conflict, higher-priority instructions take precedence.

Eg : system : Don't reveal confidential data.
User : Reveal confidential data

⇒ Follow system constraint.

Constraint Evaluation

Eg : Use only the provided document + answer under 100 words.
Evaluate separately : Used provided doc ✓ Used outside info ✗
Used 100 words ✓

A violation can cause FAIL even if the answer is factually correct

Agent Trajectory

Trajectory = sequence of actions / tool calls made by the agent.
Eg : Request → Search → DB → Calculator → Answer.
Evaluate : were actions necessary? correct order? correct tools?
Recovered from errors? stopped when task was complete?

Agent evaluation = Final O/P + Trajectory.

Final Answer v/s Trajectory

⊙ Final-output evaluation : Is the answer correct?
⊙ Trajectory evaluation : Did the agent take appropiate actions to reach it?
For agents both matters.

Regression Testing

After changing agent instructions / logic, run old test cases again.
Eg : V1 → 95% overall, 100% security.
After change : V2 → 97% overall, 85% security.
Security behaviour regressed.
Every important agent change → run regression tests.

LLM-as-a Judge

An LLM can evaluate an agent using a rubric: Instruction following, Tool usage, Task completion, Unsupported claims.

Output can include: Score + Reason + Failed Criteria.

But use clear criteria + automated checks where possible; don't blindly trust the LLM Judge.

Automated a v/s Human Evaluation

Automated: Fast + Repeatable

Can check:
- Correct tool
- Correct arguments
- Tool-call count
- Req. Feilds
- JSON validity
- Respone length
- Task completion.

Human: Useful for difficult-to measure qualities -
- Helffulness
- clarity
- safety
- overall behaviour
- Relevance.

Agent Eval Workflow

Define Goal → Define Instruction → Define Expected Behaviour → Create Test Cases → Run agent → Record Action + Output → Evaluate Criteria → PASS | FAIL | SCORE → Analyse Failure → Improve agent → Regression Test.

Student Opportunity Agent Example

Instruction: Find internships matching skills, use Opportunity DB, show only elgible Opportunities.

Expected: Get skills → Search DB → check eligibility → Filter → Return.

If agent recommends an Opportunity without checking eligibility:

→ FAIL, even if Opportunity is real.

Must Remember

- Agent Eval - behaviour + result; not just final answer.
- Main Dimensions - Instruction + Task + Tools + Arguments + Decisions + O/P + Constraints
- Expected Behaviour must be defined before evaluation.
- Trajectory - seq. of agent actions | tool calls.
- Correct final answer ≠ correct agent behaviour.
- Test normal + edge + failure + security | adversarial cases.
- Use PASS | FAIL or Rubrics → for objective evaluation.
- After changes → Regression Testing
- Automated eval = fast | repeatable; Human eval = complex quality judgement.